

Junior Software Developer

Technical Assessment: Site Safety Forms App

Overview

Thank you for your interest in joining RAS (Ron Anderson & Sons). This assessment asks you to build a small web application that emulates an internal tool: framers working on job sites complete daily safety forms (with photos), and an administrator reviews who submitted forms on each site.

Important: None of the work submitted for this assessment will be used by RAS. The goal is to emulate an internal application so we can understand your grasp of the technology and your ability to build a product end to end.

The Scenario

RAS crews work across several construction sites. Before starting work, each framer must complete a safety form for the site they are on. Supervisors need a single place to see which people submitted forms, on which sites, and when, and to review the attached photos.

Requirements

1. Tech stack & branding

- The frontend must be built with **React**. Backend, database and hosting are your choice (e.g. Node/Express, Next.js, Supabase, Firebase, PostgreSQL, etc.).
- Use the **RAS logo and RAS brand colors** throughout the app so it looks like an internal RAS product. Logo and colors are not provided: find them yourself on the **RAS website and Instagram**.
- The worker side must be usable on a phone (responsive), since forms are filled out on site.

2. Authentication & roles

- Users log in with credentials. There are at least two roles: **Framer** (worker) and **Admin**.
- Framers can only create and view their own submissions. Admins can view all submissions.

3. Safety form (Framer)

- Select the job site and date; worker name is taken from the logged-in user.
- Safety checklist, for example: PPE worn (hard hat, vest, boots, eye protection), fall protection in place, ladders/scaffolding inspected, tools and cords in good condition, hazards identified, and a free-text notes field.
- **Photo upload:** attach one or more photos (e.g. site conditions, PPE, hazards). Photos must be stored and viewable later.
- Basic validation (required fields, file type/size) and a clear success/error message.

4. Admin dashboard

- List of submissions showing worker, site, date and status.
- Filter by **site**, worker and date range; see which people submitted forms on each site.
- Detail view of a submission, including its photos.

- A simple summary (e.g. submissions per site, or who has not submitted today). Charts are a bonus.

5. Data

- If needed, you may create **fake / seed data** (sites, workers, submissions) to demonstrate that the form and the dashboard work.

Use of AI Tools

You **may use AI tools** (ChatGPT, Claude, Copilot, etc.) while building this assessment. However, it is essential that when you submit, you **fully understand how your code works**. In the follow-up interview you will be asked to walk us through your code, explain your decisions, and **debug or modify it live**. Code you cannot explain will count against you.

Deliverables

Deliverable	Details
Deployed app link	Public URL of the working app (Vercel, Netlify, Render, Railway, etc.).
GitHub repository	Link to a public repo (or private with access granted to us). Include a README with setup instructions, tech stack, and any assumptions.
Test credentials	Login credentials for at least one Admin and one Framer account so we can test both roles.
Entity-Relationship Diagram	ERD of your database (users, sites, submissions, photos, etc.) showing entities, key fields and relationships. Image or PDF, also linked in the README.

Evaluation Criteria

Area	What we look for
Functionality	Form submission, photo upload, login/roles and dashboard work as described.
Code quality	Readable structure, sensible components, naming, and error handling.
Data modeling	A clear, well-reasoned ERD that matches the implementation.
Product & UX	Branding applied, mobile-friendly form, a dashboard that is easy to use.
Understanding	Ability to explain the code and debug it live during the interview.

Submission

Reply to the email you received with: (1) deployed app link, (2) GitHub link, (3) test credentials, and (4) the ERD. Please submit within the timeframe stated in that email. If anything is unclear, make a reasonable assumption and document it in your README.

Good luck - we look forward to seeing what you build.